

CAVI-Alibori — Architecture technique complète

Stack : Laravel · Budget : tiers gratuits uniquement · Portée : architecture complète (MVP + vision)

Convention utilisée dans tout le document : **[OBLIGATOIRE]** = nécessaire pour que le pipeline fonctionne de bout en bout, **[FACULTATIF]** = améliore la robustesse/qualité mais peut être sauté pour le hackathon sans casser la démo.

1. Principes d'architecture

1. **Séparation stricte extraction (IA) / décision (règles)** — l'IA ne décide jamais seule d'une consigne agricole. Elle structure l'information ; une table de règles audité par l'ATDA prend la décision. C'est un argument de fiabilité face au jury, et c'est aussi ce qui rend le système maintenable sans réentraîner un modèle à chaque mise à jour agronomique.
 2. **Le serveur porte toute l'intelligence, le terrain ne porte que la voix** — aucun canal utilisateur final n'exige de smartphone, d'app installée, ou de compétence numérique.
 3. **Stack unique (Laravel)** — cohérent avec votre stock de compétences (internship DigiMama, Gaius déjà sur Laravel/Filament). Moins de changement de contexte, plus de vitesse d'exécution en 72h.
 4. **Chaque intégration externe a un plan de repli local** — le système ne doit jamais dépendre à 100 % d'une API tierce pour fonctionner en démo.
-

2. Stack technique — vue d'ensemble

Couche	Choix	Statut	Pourquoi
Framework backend	Laravel 11+	OBLIGATOIRE	Votre stack de référence, écosystème complet (queues, scheduler, ORM, tests)
Base de données (démon)	SQLite	OBLIGATOIRE (démon)	Zéro configuration, fichier unique, suffisant pour le volume d'un pilote sur une commune
Base de données (production)	MySQL ou PostgreSQL	FACULTATIF pour le hackathon	À migrer quand le volume/la concurrence augmentent — Laravel migre sans réécrire la logique métier
Panel admin / validation	Filament PHP	OBLIGATOIRE	Gratuit, open source, génère un CRUD complet pour que l'ATDA valide les règles sans toucher au code — déjà utilisé par Gaïus sur DigiMama
Files d'attente	Laravel Queue (driver database)	OBLIGATOIRE	Traite extraction NLP / génération audio en tâche de fond sans bloquer la requête HTTP.

Couche	Choix	Statut	Pourquoi
			Pas besoin de Redis pour ce volume
File d'attente avancée	Redis	FACULTATIF	Utile seulement au-delà de quelques centaines de messages/jour
Scheduler	Laravel Task Scheduling	OBLIGATOIRE	Récupère automatiquement les nouveaux bulletins météo à intervalle régulier
Traitement audio	FFmpeg (via <code>Symfony\Process</code>)	OBLIGATOIRE	Standard, gratuit, fiable pour concaténer les segments audio pré-enregistrés
Extraction NLP	API LLM externe + repli par règles	OBLIGATOIRE (le repli l'est ; l'API est FACULTATIF en mode dégradé)	Voir section 5
Tests	PHPUnit (Feature + Unit)	FACULTATIF mais fortement recommandé	Le moteur de règles est une logique pure, donc trivialement testable – bon signal de maturité pour le jury
Frontend admin	Blade + Filament (pas de SPA)	OBLIGATOIRE	Évite la complexité d'un

Couche	Choix	Statut	Pourquoi
	séparée)		frontend JS séparé pour un panel interne
App mobile légère (canal complémentaire)	Flutter ou simple PWA	FACULTATIF	Explicitement hors scope MVP — c'est un canal secondaire, pas la porte d'entrée principale

3. Schéma de données (cœur du système)

```

communes
├─ id, nom, departement, couverture_reseau_estimee (à
documenter), created_at

bulletins
├─ id, source (enum: meteo_benin | sap_mr | test),
texte_brut, recu_le,
| statut_extraction (enum: en_attente | extrait | echec)

variables_extraites
├─ id, bulletin_id (FK), commune_id (FK, nullable),
niveau_risque, duree_jours,
| confiance (float, nullable), methode (enum: llm |
regex_fallback),
| reponse_brute_llm (json, nullable), extrait_le

catalogue_actions
├─ id, code, description_fr, version, valide_par (FK users),
valide_le

regles_decision
├─ id, niveau_risque, duree_min, duree_max, action_id (FK
catalogue_actions),
| valide_par (FK users), valide_le, version, actif
(boolean)

```

```

segments_audio
├ id, langue (enum: bariba | peulh | dendi), type_slot
(enum: commune | risque | duree | action | connecteur),
| valeur_slot, chemin_fichier, locuteur, duree_ms

messages_generes
├ id, bulletin_id (FK), regle_id (FK), langue,
chemin_fichier_final, genere_le

journaux_diffusion
├ id, message_id (FK), canal (enum: svi | whatsapp |
radio), destinataire_ref,
| statut (enum: envoye | echec | ecoute_confirnee),
envoye_le

relais_cooperatives
├ id, nom, commune_id (FK), telephone, whatsapp_id

users (Filament)
├ id, nom, role (enum: admin | agronome_atda | dev)

```

Pourquoi ce découpage : chaque table correspond à une étape auditable du pipeline. Si le jury demande "pourquoi cette consigne a été choisie ?", vous pouvez remonter `messages_generes` → `regles_decision` → `variables_extraites` → `bulletins` sans ambiguïté. C'est la traçabilité que vous avez déjà promise dans le pitch (section 7, limite n°3).

4. Pipeline — étapes et responsabilités

4.1 Ingestion [OBLIGATOIRE]

```

// app/Console/Commands/FetchBulletinCommand.php
// Programmé via app/Console/Kernel.php : -
>everyThirtyMinutes()
class FetchBulletinCommand extends Command
{
    public function handle(BulletinFetcherService $fetcher)
    {
        $bulletin = $fetcher->fetchLatest(); // scrape ou
API si disponible

```

```

        Bulletin::create([...]);
        ExtractVariablesJob::dispatch($bulletin);
    }
}

```

Point de vigilance déjà identifié : la granularité réelle des bulletins (commune vs département) n'est pas confirmée — à vérifier avant de coder cette brique, pas après.

4.2 Extraction NLP [le mécanisme est OBLIGATOIRE, le fournisseur LLM est remplaçable]

Architecture en interface + adaptateurs (pattern Strategy) — permet de changer de fournisseur sans toucher au reste du système :

```

interface NlpExtractorInterface
{
    public function extract(string $texteBulletin):
    ExtractionResult;
}

class LlmExtractor implements NlpExtractorInterface { /*
appel API */ }
class RegexFallbackExtractor implements
NlpExtractorInterface { /* règles locales */ }

```

```

class ExtractVariablesJob implements ShouldQueue
{
    public function handle(NlpExtractorInterface $extractor)
    {
        try {
            $result = $extractor->extract($this->bulletin->
            >texte_brut);
        } catch (ApiException $e) {
            $result = app(RegexFallbackExtractor::class)->
            >extract($this->bulletin->texte_brut);
        }
        VariableExtraite::create([...]);
        ApplyDecisionRuleJob::dispatch($result);
    }
}

```

Ce pattern répond directement à la question déjà anticipée dans votre pitch ("et si l'API tombe pendant la démo ?") — le fallback n'est pas une astuce de présentation, il est câblé dans le code.

4.3 Moteur de décision [OBLIGATOIRE]

Logique pure, sans dépendance externe — donc la plus facile à tester unitairement :

```
class DecisionEngine
{
    public function decide(ExtractionResult $r): ?
    RegleDecision
    {
        return RegleDecision::query()
            ->where('actif', true)
            ->where('niveau_risque', $r->niveauRisque)
            ->whereBetween('duree_jours', [$r->dureeMin, $r->dureeMax])
            ->first();
    }
}
```

Test associé (à écrire, facultatif mais valorisant) :

```
public function
test_secheresse_severe_dix_jours_selectionne_paillage()
{
    $regle = RegleDecision::factory()->create([...]);
    $resultat = (new DecisionEngine)->decide(new
    ExtractionResult(...));
    $this->assertEquals($regle->action->code, $resultat->action->code);
}
```

4.4 Génération audio (Slot-Filling) [OBLIGATOIRE]

```
class AudioAssemblyService
{
    public function assemble(RegleDecision $regle, Commune
    $commune, string $langue): string
```

```

{
    $segments = [
        SegmentAudio::forSlot('commune', $commune->nom,
$langue)->chemin_fichier,
        SegmentAudio::forSlot('risque', $regle->
niveau_risque, $langue)->chemin_fichier,
        SegmentAudio::forSlot('action', $regle->action-
>code, $langue)->chemin_fichier,
    ];
    $sortie = storage_path("app/audio/genere_" .
uniqid() . ".mp3");
    // Concaténation via ffmpeg (concat demuxer)
    Process::run("ffmpeg -f concat -safe 0 -i {$this-
>buildConcatFile($segments)} -c copy {$sortie}");
    return $sortie;
}
}

```

Point de vigilance déjà identifié : tester tôt la fluidité aux jonctions, sur le scénario unique retenu pour la démo — pas sur l'ensemble de la matrice commune × risque × durée × action.

4.5 Diffusion [le mécanisme est OBLIGATOIRE, chaque canal individuel est négociable]

Même pattern Strategy que pour l'extraction :

```

interface CanalDiffusionInterface
{
    public function diffuser(MessageGenere $message, array
$destinataires): void;
}

class SviChannel implements CanalDiffusionInterface { /*
endpoint webhook consulté par le fournisseur télécom */ }
class WhatsAppChannel implements CanalDiffusionInterface {
/* Meta Cloud API */ }
class RadioChannel implements CanalDiffusionInterface { /*
export fichier + notification manuelle, pas d'API */ }

```

5. Intégrations externes — choix concrets pour un budget gratuit

5.1 Extraction NLP — fournisseur LLM

Recommandation : Groq (inférence Llama, free tier généreux, très rapide) ou Google Gemini (free tier généreux, sortie JSON structurée native). Les deux imposent une sortie JSON stricte (`response_format / response_schema`), ce qui simplifie l'intégration avec `ExtractionResult`.
Fait à vérifier avant implémentation : les conditions des tiers gratuits changent fréquemment — reconfirmer les quotas au moment de coder, pas en se fiant à ce document dans plusieurs semaines.

5.2 SVI (téléphonie vocale)

Pour la démo hackathon [OBLIGATOIRE pour rester réaliste] : simuler le SVI via une interface web (mockup de téléphone dans Filament ou une page Blade dédiée) qui joue le fichier audio généré. Zéro dépendance externe, zéro risque de panne en direct — et c'est déjà ce que votre propre pitch prévoit ("SVI simulé ou fonctionnel").

Pour la feuille de route post-hackathon [FACULTATIF, à valider] : Africa's Talking propose une API Voice pensée pour l'Afrique, avec un compte développeur gratuit — mais sa couverture pays n'est pas confirmée pour le Bénin dans la documentation publique (elle couvre clairement le Kenya, le Nigeria, l'Ouganda, la Tanzanie, l'Afrique du Sud, le Ghana ; le Bénin n'apparaît pas explicitement). **À vérifier directement avec eux avant tout engagement** plutôt que de le présenter comme acquis au jury. Alternative : Twilio (couverture mondiale, mais crédit d'essai limité dans le temps, pas gratuit en continu).

5.3 WhatsApp

Recommandation : l'API officielle Meta Cloud, qui fournit un numéro de test et des destinataires de test sans vérification d'entreprise — utilisable immédiatement et gratuitement pour la démo. La vérification business (nécessaire pour un vrai déploiement) prend du temps et sort du scope des 72h.

5.4 Radio communautaire

Pas d'API : c'est un export de fichier audio + une notification (email/WhatsApp) au partenaire radio. La complexité ici est humaine (partenariat), pas technique — ne pas sur-ingénierer cette brique.

5.5 Hébergement gratuit

Le point d'attention : un canal SVI a besoin d'un endpoint **toujours joignable** (le fournisseur télécom appelle votre webhook). Beaucoup d'hébergements gratuits (Render free tier, par exemple) mettent l'application en veille après inactivité — acceptable pour une démo pilotée par vous, risqué pour une disponibilité réelle.

- **Pour la démo** : Render ou Railway (déploiement Laravel simple, mise en veille tolérable puisque vous contrôlez le timing).
- **Pour une disponibilité réelle post-hackathon [FACULTATIF]** : Oracle Cloud Free Tier propose une VM "always free" à vie, suffisante pour un pilote sur une commune — plus de configuration initiale, mais pas de veille ni de limite de temps.

Fait à vérifier avant implémentation : les offres d'hébergement gratuit évoluent souvent — reconfirmer les conditions actuelles avant de s'engager.

6. Sécurité et gouvernance [mélange OBLIGATOIRE / FACULTATIF selon l'élément]

- **Traçabilité des règles** [OBLIGATOIRE] — chaque `regle_decision` a un `valide_par` et un `valide_le` : impossible de diffuser une consigne non validée par un humain compétent.
- **Clés API hors code** [OBLIGATOIRE] — fichier `.env`, jamais commité, quel que soit le fournisseur choisi.
- **Journal d'audit des diffusions** [OBLIGATOIRE] — `journaux_diffusion` permet de répondre à "combien de personnes ont reçu quoi, quand".
- **Consentement et protection des données pour la boucle de retour vocal** [FACULTATIF — vision long terme uniquement] — à traiter

seulement quand cette fonctionnalité sera réellement développée, pas en MVP.

- **Rate limiting sur les endpoints publics (webhook SVI/WhatsApp)**
[FACULTATIF pour la démo, recommandé en production] — Laravel le fournit nativement (`throttle` middleware), coût d'implémentation minime.
-

7. Ce qui est strictement nécessaire pour la démo de 72h

Périmètre minimal fonctionnel, aligné sur le scope déjà validé (une commune, un scénario, un canal) :

1. Migration + modèles Eloquent pour les 8 tables du cœur (section 3, sans `relais_cooperatives` si le canal retenu est le SVI simulé)
2. Une commande Artisan qui ingère un bulletin de test (fichier texte local, pas besoin de scraping live pour la démo)
3. `LlmExtractor` + `RegexFallbackExtractor` fonctionnels
4. `DecisionEngine` avec 2-3 règles pré-validées manuellement en base
5. `AudioAssemblyService` avec une dizaine de segments enregistrés (assez pour un scénario)
6. Une page Blade simulant l'appel SVI qui joue le message généré
7. Panel Filament minimal pour montrer la validation des règles par un "agronome"

Tout le reste (Redis, MySQL, WhatsApp réel, hébergement always-on, tests exhaustifs) est de la robustesse pour l'après-hackathon, pas un prérequis de démonstration.

8. Points de vigilance sur cette architecture elle-même

Trois faiblesses à connaître avant de s'engager dessus :

1. Le pattern Strategy (interfaces + adaptateurs) a un coût d'abstraction

— pour un prototype de 72h, ça peut sembler sur-ingénierié. Je le recommande quand même parce que le repli LLM → regex et le canal simulé → réel sont des besoins déjà identifiés, pas hypothétiques — mais si le temps manque, une version sans interface (un simple `if/else` sur un flag de config) atteint le même résultat plus vite, au prix d'un refactor plus tard.

2. SQLite en démo peut masquer des problèmes de concurrence — si plusieurs jurés testent le SVI simulé en même temps, SQLite gère mal les écritures concurrentes. Pour une démo à un seul flux à la fois, ce n'est pas un risque réel ; pour un vrai pilote multi-commune, la migration vers MySQL/PostgreSQL devient nécessaire tôt.

3. Aucune de ces briques ne résout les risques déjà identifiés hors architecture — granularité des bulletins, couverture réseau, coût du numéro vert, incertitude météo. Une architecture "en béton" peut très bien tourner sur des hypothèses de terrain non vérifiées. Le code ne remplace pas la vérification terrain listée dans le doc de synthèse.